

Hardware Design and FPGA Implementation of a Polynomial Multiplication Accelerator for CRYSTALS-Dilithium

Linh Tran

*Faculty of Electrical and Electronics Engineering
Ho Chi Minh City University of Technology, VNU-HCM
Ho Chi Minh City, Vietnam
linhtran@hcmut.edu.vn*

Hoang Anh

*Faculty of Electrical and Electronics Engineering
Ho Chi Minh City University of Technology, VNU-HCM
Ho Chi Minh City, Vietnam
anh.phamhoanganh@hcmut.edu.vn*

Hieu Le

*Faculty of Electrical and Electronics Engineering
Ho Chi Minh City University of Technology, VNU-HCM
Ho Chi Minh City, Vietnam
hieu.le30042004@hcmut.edu.vn*

Abstract—Polynomial multiplication is a dominant arithmetic kernel in lattice-based post-quantum cryptography, and its hardware cost strongly affects the performance of lattice-based signature schemes. This paper presents a scalable FPGA accelerator for CRYSTALS-Dilithium polynomial multiplication with 256 coefficients and the Dilithium modulus 8380417. Instead of implementing an isolated transform core, the proposed accelerator executes the complete computation flow, including forward transforms for the two input polynomials, pointwise Montgomery multiplication, inverse transform, and final scaling. The architecture is organized as a family of four-, eight-, and sixteen-butterfly-unit configurations that share the same control template, memory-banking policy, twiddle-factor access pattern, and arithmetic datapath. The design combines schedule-ROM based address generation, banked polynomial memories, lane-aligned zeta ROMs, a unified forward and inverse butterfly datapath, and pipeline-aligned writeback metadata. To reduce multiplier cost for the 23-bit Dilithium modulus, the Montgomery multiplication datapath exploits sparse constant forms so that only the main coefficient product is mapped to DSP hardware while reduction-constant products are realized by shift, add, and subtract networks. The resulting architecture provides a consistent basis for studying the area-throughput trade-off of Dilithium polynomial multiplication on FPGA and is verified against a coefficient-accurate golden reference model.

Index Terms—CRYSTALS-Dilithium, polynomial multiplication, NTT, INTT, Montgomery multiplication, FPGA, hardware accelerator.

I. INTRODUCTION

Lattice-based cryptography has become one of the leading directions for post-quantum cryptographic systems. CRYSTALS-Dilithium, standardized as ML-DSA in FIPS 204, relies on arithmetic over polynomial rings. Polynomial multiplication is repeatedly used inside polynomial-vector and matrix-vector operations; therefore, its latency has a direct impact on the cost of a complete signature implementation. A direct schoolbook implementation has quadratic complexity.

In contrast, an NTT-based implementation reduces polynomial multiplication to two forward transforms, a coefficient-wise product, and one inverse transform:

$$C = \text{INTT}(\text{NTT}(A) \circ \text{NTT}(B)), \quad (1)$$

where $\circ$ denotes pointwise Montgomery multiplication.

Previous hardware works show that NTT accelerators are limited not only by arithmetic units but also by memory access. Several Kyber accelerators report low-latency NTT, INTT, or polynomial multiplication cores under a 12-bit modulus [9], [10]. Other lattice-cryptography accelerators target different moduli, software platforms, or transform-only datapaths [3]–[7]. These works motivate the use of unified butterfly datapaths, twiddle-factor ROMs, and careful memory scheduling. However, Dilithium uses the 23-bit modulus 8380417, and a full Dilithium polynomial multiplication datapath must preserve the exact order of forward transforms, pointwise Montgomery multiplication, inverse transform, and final scaling. Therefore, the design problem is not only to make an NTT core fast, but also to keep memory banking, zeta ordering, writeback metadata, and reduction arithmetic aligned across the complete flow.

This work presents a complete Dilithium polynomial multiplication accelerator rather than an isolated NTT core. The novelty is the integration of a lane-scalable butterfly engine with a conflict-free banked memory system and delayed writeback metadata for the full Dilithium multiplication sequence. The design is developed in three scalable configurations: 4BU, 8BU, and 16BU. All configurations share the same cycle-level architecture, including arithmetic lanes, memory policy, schedule-ROM structure, top-level phase sequencing, and verification method. Therefore, the difference among 4BU, 8BU, and 16BU mainly reflects the effect of butterfly parallelism and memory bandwidth instead of unrelated design choices.

TABLE I
MAIN CONTRIBUTIONS OF THIS WORK.

Item	Contribution
C1	Complete Dilithium polynomial multiplication accelerator, not only an isolated transform core: NTT(A), NTT(B), pointwise Montgomery, INTT, and final scaling.
C2	Scalable 4BU/8BU/16BU architecture family using the same cycle-level scheduler, banked memory policy, butterfly datapath, and verification methodology.
C3	Unified arithmetic lane for Cooley–Tukey NTT, Gentleman–Sande INTT, pointwise multiplication, and final scaling.
C4	DSP-efficient Montgomery datapath for the 23-bit Dilithium modulus by mapping sparse reduction-constant products to shift/add/sub networks.
C5	Schedule-ROM and 32-bank memory organization with explicit conflict-free bank-row metadata and pipeline-aligned writeback tags.
C6	Layered verification from arithmetic units to full 256-coefficient end-to-end polynomial multiplication on both simulation and FPGA-oriented self-test flows.

The proposed design is intended as a controlled architecture study for Dilithium polynomial multiplication. Instead of reporting only one optimized point, it exposes how latency, memory usage, DSP count, ALM/LUT count, register count, and Fmax change when the number of butterfly lanes is scaled. This makes the design useful for selecting a compact accelerator, a balanced accelerator, or a high-throughput accelerator under a fixed FPGA resource budget.

II. ALGORITHMIC BACKGROUND

This section introduces the notation and arithmetic background used throughout the paper. The presentation follows the structure commonly used in NTT accelerator papers: first the ring notation is defined, then the NTT-based multiplication formula is introduced, and finally the Dilithium-specific arithmetic parameters are discussed.

A. Notation

Let $\mathbb{Z}_q$ denote the ring of integers modulo q , i.e.,

$$\mathbb{Z}_q = \{0, 1, \dots, q-1\}. \quad (2)$$

For CRYSTALS-Dilithium, the polynomial ring is

$$R_q = \mathbb{Z}_q[x]/(x^n + 1), \quad (3)$$

where

$$n = 256, \quad q = 8380417. \quad (4)$$

A polynomial $a(x) \in R_q$ is represented as

$$a(x) = \sum_{i=0}^{n-1} a_i x^i, \quad a_i \in \mathbb{Z}_q. \quad (5)$$

The coefficient vector form is written as

$$a = [a_0, a_1, \dots, a_{255}]. \quad (6)$$

In this paper, $\bar{a}$ denotes the NTT-domain representation of a , and $\circ$ denotes coefficient-wise multiplication in the NTT domain.

The transform uses twiddle factors in Montgomery representation. A forward twiddle factor is denoted by ζ , and an inverse twiddle factor is denoted by ζ^{-1} . The Montgomery reduction operation is denoted by $\text{MR}(\cdot)$. The final inverse scaling constant used in Dilithium is

$$f = 41978. \quad (7)$$

Thus, after the inverse transform, each coefficient is multiplied by f through the same Montgomery multiplication datapath.

B. NTT-Based Polynomial Multiplication

The NTT is the finite-field analogue of the discrete Fourier transform. It maps a polynomial from coefficient representation to a transform-domain representation where convolution becomes coefficient-wise multiplication. For two polynomials $a, b \in R_q$, the schoolbook multiplication has quadratic complexity. NTT-based multiplication reduces this operation to

$$c = \text{INTT}(\text{NTT}(a) \circ \text{NTT}(b)). \quad (8)$$

Equation (8) is the algorithmic basis of the proposed accelerator.

The forward transform used by the hardware follows a Cooley–Tukey (CT) in-place butterfly. For a coefficient pair (A, B) and twiddle factor W , the CT butterfly computes

$$T = \text{MR}(B \cdot W), \quad (9)$$

$$E = A + T \pmod{q}, \quad (10)$$

$$O = A - T \pmod{q}. \quad (11)$$

The inverse transform follows a Gentleman–Sande (GS) in-place butterfly:

$$E = A + B \pmod{q}, \quad (12)$$

$$D = A - B \pmod{q}, \quad (13)$$

$$O = \text{MR}(D \cdot W). \quad (14)$$

Equations (9)–(14) are important because they show that CT and GS require the same primitive operators: modular addition, modular subtraction, multiplication, and Montgomery reduction. The proposed butterfly unit exploits this overlap.

The complete multiplication schedule is

$$\bar{a} = \text{NTT}(a), \quad \bar{b} = \text{NTT}(b), \quad (15)$$

$$\bar{c}_i = \text{MR}(\bar{a}_i \cdot \bar{b}_i), \quad 0 \leq i < n, \quad (16)$$

$$c = \text{INTT}(\bar{c}), \quad c_i \leftarrow \text{MR}(f \cdot c_i). \quad (17)$$

The proposed accelerator implements all these steps internally; therefore, the reported latency is full polynomial multiplication latency rather than isolated NTT latency.

Algorithm 1. Forward in-place NTT for Dilithium

Input: $a[0 \dots 255]$, forward twiddle table ζ
Output: $a[0 \dots 255]$ in NTT domain

```
1:  $k \leftarrow 1$ 
2: for  $len \leftarrow 128, 64, \dots, 1$  do
3:   for  $start \leftarrow 0$  to  $255$  step  $2len$  do
4:      $W \leftarrow \zeta[k], k \leftarrow k + 1$ 
5:     for  $j \leftarrow start$  to  $start + len - 1$  do
6:        $A \leftarrow a[j], B \leftarrow a[j + len]$ 
7:        $T \leftarrow MR(B \cdot W)$ 
8:        $a[j] \leftarrow A + T \pmod{q}$ 
9:        $a[j + len] \leftarrow A - T \pmod{q}$ 
10:    end for
11:  end for
12: end for
```

Algorithm 2. Inverse in-place NTT and final scaling

Input: $a[0 \dots 255]$, inverse twiddle table ζ^{-1}
Output: $a[0 \dots 255]$ in coefficient domain

```
1:  $k \leftarrow 255$ 
2: for  $len \leftarrow 1, 2, \dots, 128$  do
3:   for  $start \leftarrow 0$  to  $255$  step  $2len$  do
4:      $W \leftarrow \zeta^{-1}[k], k \leftarrow k - 1$ 
5:     for  $j \leftarrow start$  to  $start + len - 1$  do
6:        $A \leftarrow a[j], B \leftarrow a[j + len]$ 
7:        $a[j] \leftarrow A + B \pmod{q}$ 
8:        $a[j + len] \leftarrow MR((A - B) \cdot W)$ 
9:     end for
10:  end for
11: end for
12: for  $j \leftarrow 0$  to  $255$  do
13:    $a[j] \leftarrow MR(41978 \cdot a[j])$ 
14: end for
```

C. Dilithium Arithmetic Parameters

Dilithium differs from Kyber in the size of its coefficient modulus. Kyber uses a 12-bit modulus $q = 3329$, whereas Dilithium uses a 23-bit modulus $q = 8380417$ [1], [2]. This difference directly affects hardware cost because adders, subtractors, multipliers, and reduction units must process wider values.

The modulus and Montgomery constant used in this work have sparse binary forms:

$$q = 8380417 = 2^{23} - 2^{13} + 1, \quad (18)$$

$$QINV = 58728449 = 2^{25} + 2^{24} + 2^{23} + 2^{13} + 1. \quad (19)$$

Equations (18) and (19) are exploited in the proposed Montgomery reduction unit. Instead of implementing all constant products with DSP multipliers, the constant products are mapped to shift/add/sub networks. This is one of the main architectural optimizations of the design.

III. PROPOSED DESIGN

This section presents the main design choices of the proposed polynomial multiplication accelerator. Following the style of prior NTT accelerator papers [9], [10], the discussion is organized around the optimized hardware blocks rather than around low-level RTL signals. The highlighted blocks are the Montgomery reduction unit, the unified Cooley–Tukey (CT) and Gentleman–Sande (GS) butterfly lane, the schedule-ROM address generator, the conflict-free banking scheme,

Algorithm 3. Full polynomial multiplication schedule

Input: Polynomials $A[0 \dots 255]$ and $B[0 \dots 255]$
Output: Polynomial $C = A \cdot B$

```
1: Load  $A$  into memory A and  $B$  into memory B
2: Run forward NTT on memory A
3: Run forward NTT on memory B
4: for  $i \leftarrow 0$  to  $255$  do
5:    $A[i] \leftarrow MR(A[i] \cdot B[i])$ 
6: end for
7: Run inverse NTT and final scaling on memory A
8: Read  $C$  from memory A
```

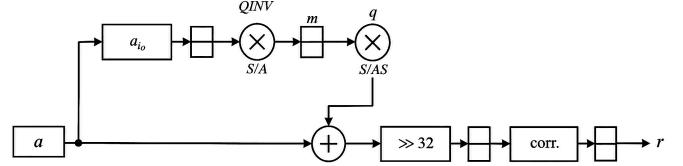


Fig. 1. Montgomery reduction unit.

the clustered butterfly array, and the compact full-PMUL organization.

A. Montgomery Reduction Unit

The modular multiplication datapath is designed around Montgomery reduction for the Dilithium modulus $q = 8380417$. A product of two 23-bit coefficients produces a wide intermediate value. A direct implementation of the Montgomery equations may introduce several multipliers in the reduction path, which would make the DSP count grow faster than the number of butterfly lanes. In the proposed design, only the main coefficient product is mapped to a DSP block, while the constant products inside the reduction unit are implemented by shift/add/sub networks.

For a product $a = x \cdot y$, the reduction computes

$$m = a_{lo} \cdot QINV \pmod{2^{32}}, \quad (20)$$

$$t = (a - mq) \gg 32, \quad (21)$$

$$r = \text{correction}(t). \quad (22)$$

The sparse constants $q = 2^{23} - 2^{13} + 1$ and $QINV = 2^{25} + 2^{24} + 2^{23} + 2^{13} + 1$ allow the products $a_{lo} \cdot QINV$ and $m \cdot q$ to be implemented without general-purpose multipliers. Therefore, the reduction unit in Fig. 1 uses a shift/add block for $QINV$ and a shift/add/sub block for q .

Figure 1 shows the internal datapath of the MR unit. S/A denotes a shift/add network and S/A/S denotes a shift/add/sub network. These two logic networks replace the constant multipliers associated with $QINV$ and q .

In Fig. 1, a_{lo} denotes the lower 32 bits of the product a , m is the Montgomery correction factor, $\gg 32$ denotes the final Montgomery shift, and $corr.$ denotes conditional correction into the target coefficient range. The thin vertical symbols represent pipeline boundaries used to align the long reduction path.

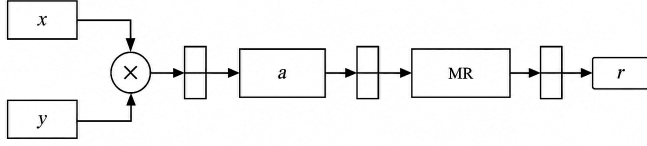


Fig. 2. Modular multiplier.

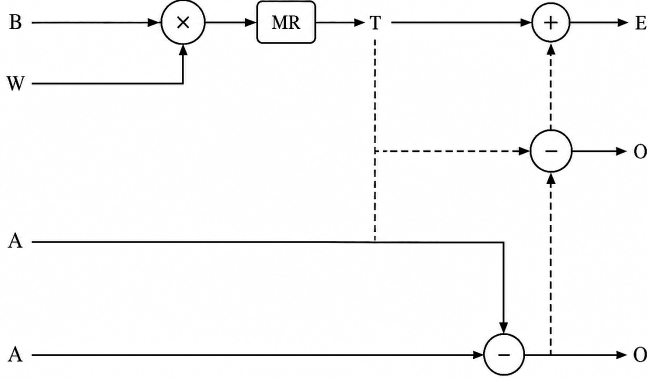


Fig. 3. Cooley-Tukey butterfly.

The complete modular multiplier is shown in Fig. 2. The multiplier first computes $a = x \cdot y$ using one DSP block. The following MR unit performs the modular reduction using the logic-based structure in Fig. 1. This separation is important for scalability: a P -butterfly implementation uses exactly P DSP blocks for the P coefficient multipliers.

Figure 2 gives the higher-level modular multiplier built around the MR unit. The $x \times y$ product is the only operation mapped to a DSP block; the following MR block performs reduction with the logic structure of Fig. 1.

The contribution of this block is not the use of Montgomery reduction alone, but the controlled DSP cost of the reduction datapath. This gives a clean scaling rule across the three architectures: 4BU, 8BU, and 16BU use 4, 8, and 16 DSP blocks, respectively.

B. Unified Cooley-Tukey and Gentleman-Sande Butterfly

The transform engine uses CT butterflies for the forward NTT and GS butterflies for the inverse NTT. These two forms are shown separately before introducing the unified lane. In the CT form, the twiddle multiplication is performed before the modular addition and subtraction. In the GS form, the sum and difference are computed first, and only the difference path is multiplied by the inverse twiddle factor.

In Fig. 3, the multiplier branch first computes $T = \text{MR}(BW)$. The two output branches then produce the CT pair $E = A + T \pmod{q}$ and $O = A - T \pmod{q}$.

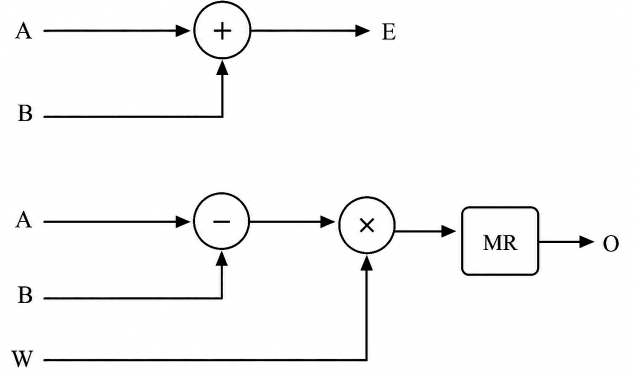


Fig. 4. Gentleman-Sande butterfly.

In Fig. 4, the upper branch computes the modular sum $E = A + B \pmod{q}$. The lower branch computes the difference first and then multiplies it by the inverse twiddle factor, giving $O = \text{MR}((A - B)W)$.

The unified butterfly in Fig. 5 merges the CT and GS structures and additionally supports pointwise Montgomery multiplication and final scaling. The mode signal selects the multiplier input X , the second operand Y , and the output pair. The four supported modes are CT, GS, PW, and SC, corresponding to forward NTT, inverse NTT, pointwise multiplication, and scaling. The shared multiplier branch computes

$$T = \text{MR}(X \cdot Y), \quad (23)$$

where X and Y are selected by the two input MUXes.

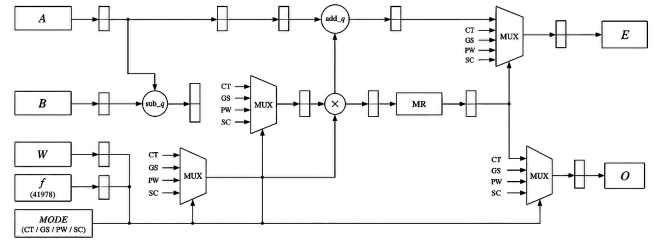


Fig. 5. Unified butterfly datapath.

Mode	Multiplier Input (X)	Operand (Y , 2nd input)	$T = \text{MR}(X \cdot Y)$	Adder Output (E path)	Output O
CT (Cooley-Tukey)	B	W	$\text{MR}(B \cdot W)$	$A + T \pmod{q}$	$A - T \pmod{q}$
GS (Gentleman-Sande)	$A - B$	W	$\text{MR}((A - B) \cdot W)$	$A + B \pmod{q}$	$\text{MR}((A - B) \cdot W)$
PW (Pointwise)	A	B	$\text{MR}(A \cdot B)$	$\text{MR}(A \cdot B)$	$\text{MR}(A \cdot B)$ or unused
SC (Scaling)	A	f (41978)	$\text{MR}(A \cdot f)$	$\text{MR}(A \cdot f)$	$\text{MR}(A \cdot f)$ or unused

Fig. 6. Unified butterfly mode selection.

In Fig. 5, add_q and sub_q denote modular addition and modular subtraction. The four MUXes are the central reuse mechanism of the lane. The first MUX selects the multiplier input X , the second MUX selects the operand Y , and the two output MUXes select the values returned as E and O . The

mode signal selects one of four datapath interpretations: CT for forward NTT, GS for inverse NTT, PW for coefficient-wise multiplication, and SC for final scaling. The small vertical rectangles are pipeline registers used for timing alignment; they should be read as datapath boundaries rather than as extra mathematical operations.

The exact MUX behavior is summarized in Fig. 6. In CT mode, the lane first computes $T = \text{MR}(BW)$ and then returns $E = A + T$ and $O = A - T$. In GS mode, the upper output is the modular sum $A + B$, while the lower output is the reduced product $\text{MR}((A - B)W)$. In PW mode, the same multiplier branch computes $\text{MR}(AB)$, which is stored back as the pointwise product. In SC mode, the operand Y is the Dilithium scaling constant $f = 41978$, so the lane computes $\text{MR}(Af)$. This table is useful because it makes clear that no separate pointwise or scaling engine is required.

This unified structure is a main reuse point of the accelerator. It avoids separate forward, inverse, pointwise, and scale datapaths. The cost is additional selection logic, but the benefit is a single pipelined arithmetic lane that can be replicated in the 4BU, 8BU, and 16BU designs.

C. Schedule-ROM Address Generation

The NTT/INTT access pattern is deterministic for a fixed transform length and banking scheme. Instead of recomputing all bank, row, and twiddle addresses with wide stage-dependent combinational logic, the proposed design stores the precomputed issue schedule in ROM. At runtime, the scheduler only provides the stage index s and issue index i .

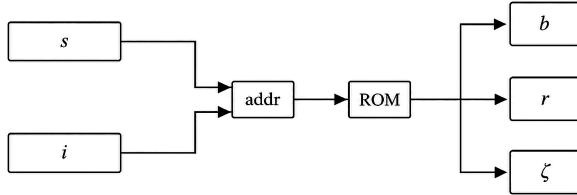


Fig. 7. Schedule ROM interface.

Figure 7 shows the compact ROM interface. The ROM output provides the bank index b , row index r , and twiddle-factor address ζ for an issue step. Here, b and r are not data values; they are memory-access metadata that select which physical bank and row are read by each butterfly lane.

Figure 8 expands the ROM output fields used by the scheduler. The pair (b, r) represents the physical bank and row for memory access, ζ is the twiddle-factor address, and μ is the writeback metadata delayed along the butterfly pipeline.

The extended schedule generator is shown in Fig. 8. The symbol μ denotes delayed writeback metadata, including the destination bank/row and write-enable information after the butterfly pipeline latency. This metadata is carried in parallel with the coefficient data so that the output of the butterfly array is written back to the correct memory location.

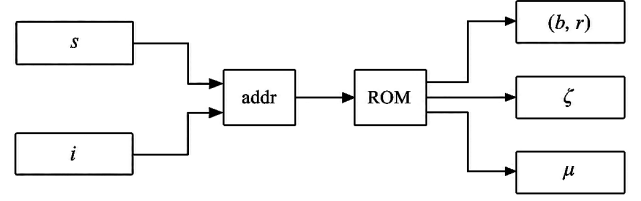


Fig. 8. Address schedule generator.

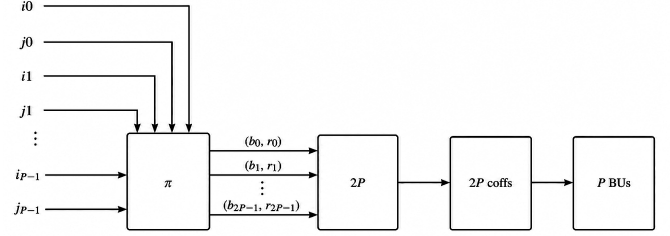


Fig. 9. Conflict-free bank mapping.

For P butterfly units, one issue cycle reads $2P$ coefficients. Therefore, the schedule must satisfy

$$b_u \neq b_v, \quad u \neq v, \quad (24)$$

for all coefficients accessed in the same cycle. This conflict-free condition is checked when the schedule ROM is generated and again in simulation.

D. Conflict-Free Banking and Clustered Butterfly Array

The polynomial memory is divided into $2P$ banks so that P butterfly pairs can be issued in one cycle. Since each polynomial contains 256 coefficients, the number of rows per bank is

$$\text{rows} = \frac{256}{2P}. \quad (25)$$

Thus, the 4BU, 8BU, and 16BU designs use 8, 16, and 32 banks per polynomial, respectively. Their corresponding row depths are 32, 16, and 8.

In Fig. 9, the input set represents the $2P$ logical coefficient indices required by one issue step. The mapping π produces the physical addresses $(b_0, r_0), \dots, (b_{2P-1}, r_{2P-1})$. The schedule is generated so that all $2P$ bank indices are distinct in that cycle. The $2P$ -bank memory can therefore return $2P$ coefficients without a bank conflict, allowing the P butterfly units to operate in parallel.

A second memory-side optimization is clustering. A monolithic P -lane butterfly array creates long operand distribution and writeback paths when P is large. The proposed implementation divides the array into two local clusters. The 4BU, 8BU, and 16BU architectures therefore use 2×2 , 2×4 , and 2×8 butterfly clusters, respectively.

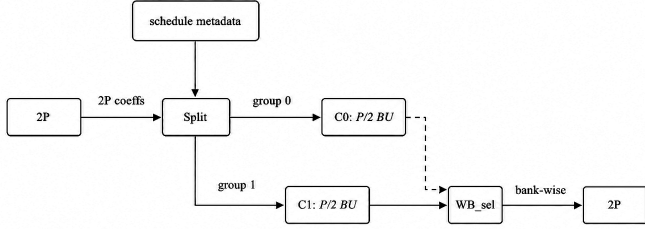


Fig. 10. Clustered butterfly array.

In Fig. 10, *Split* denotes the fixed distribution network that sends the scheduled operands to clusters *C0* and *C1*. The block *WB_sel* denotes the bank-wise writeback selector. The dashed path indicates that schedule metadata accompanies the coefficient results so that writeback remains deterministic. This organization does not change the NTT algorithm; it makes the physical datapath more regular and reduces the need for a large dynamic crossbar.

E. Overall Full-PMUL Organization

The complete accelerator combines the optimized blocks above into the full polynomial multiplication flow. Unlike an isolated NTT core, the proposed accelerator executes the complete sequence $\text{NTT}(A)$, $\text{NTT}(B)$, pointwise Montgomery multiplication, INTT, and final scaling.

Figure 11 summarizes the proposed full-PMUL datapath. The control FSM receives the start command and completion feedback, then generates ROM addresses, butterfly mode selection, BRAM controls, and writeback controls. The two input MUXes load the input polynomials into BRAM A and BRAM B. The S-ROM provides the schedule metadata, while the ζ -ROM provides forward or inverse twiddle factors. The *P*-BU array performs all arithmetic phases. The writeback MUX selects BRAM A during $\text{NTT}(A)$, pointwise multiplication, and INTT, while BRAM B is selected during $\text{NTT}(B)$. The final polynomial *C* is stored in BRAM A and read through the output port.

IV. SCALABLE 4BU, 8BU, AND 16BU DESIGNS

The proposed accelerator is implemented in three configurations. The parameter *P* denotes the number of butterfly units issued per cycle. Increasing *P* reduces the number of issue steps per NTT/INTT stage but increases memory width, schedule width, and routing pressure.

The 4BU design is the compact point. It has the smallest memory and writeback network and achieves the highest *F*_{max} among the three designs. The 8BU design is the balanced point because it halves the ideal issue count per stage compared with 4BU while keeping routing pressure moderate. The 16BU design is the throughput point. It has the smallest cycle count but requires the widest banked memory and the largest schedule words.

TABLE II
ARCHITECTURAL SCALING PARAMETERS.

Parameter	4BU	8BU	16BU
Butterfly units <i>P</i>	4	8	16
Clusters	2×2	2×4	2×8
Banks / polynomial	8	16	32
Rows / bank	32	16	8
Issue steps / stage	32	16	8
Schedule entries	256	128	64
DSP blocks	4	8	16

The important feature of the comparison is that the three designs share the same arithmetic lane, schedule-ROM principle, banking rule, and full-PMUL phase sequence. Therefore, the reported differences mainly reflect the effect of butterfly parallelism rather than unrelated architectural changes.

V. VERIFICATION SUMMARY

The design was verified using a layered test strategy. The golden model is independent from the RTL and follows the Dilithium arithmetic flow, including Montgomery reduction, forward zeta ordering, inverse zeta ordering, and final inverse scaling. Arithmetic units, butterfly wrappers, schedule ROMs, banked memories, individual NTT/pointwise/INTT phases, and the complete accelerator were tested separately. The final full-accelerator test reads the output in logical coefficient order and compares all 256 coefficients against the golden result; no approximate tolerance is used because Dilithium arithmetic is bit-accurate modulo *q*. All three configurations pass the full 256-coefficient comparison. This layered verification is important because a banked memory can still produce numerically valid coefficients while placing them in wrong logical positions.

VI. IMPLEMENTATION RESULTS AND COMPARISON

The proposed designs are evaluated on two FPGA flows. The Intel flow uses Quartus Prime 24.1 Standard Edition on Cyclone V 5CSXFC6D6F31C6, with post-fit resource usage and timing from the Quartus reports. The Xilinx flow uses Vivado 2020.2 on Kintex-7 XC7K70TFBV676-1, with post-implementation resource usage and routed timing from the Vivado reports. Both flows use a single clock constrained to 10 ns. The cycle-level RTL architecture is kept the same across the two flows; only vendor-specific project files, constraints, and memory/DSP inference behavior differ. Table III reports both sets of implementation results. The Quartus rows use ALM, register, M10K RAM block, and Intel DSP units, whereas the Vivado rows use LUT, FF, BRAM, and DSP48 units; therefore, the numbers are intended for platform-aware comparison rather than direct primitive-to-primitive equivalence.

The DSP usage grows linearly with the number of butterfly units in both flows, but the scaling factor is platform-dependent. On Cyclone V, the optimized Montgomery multiplication datapath uses one Intel DSP block per butterfly lane. On Kintex-7, Vivado maps each 23-bit by 23-bit product to two DSP48 blocks, resulting in 8, 16, and 32 DSP blocks

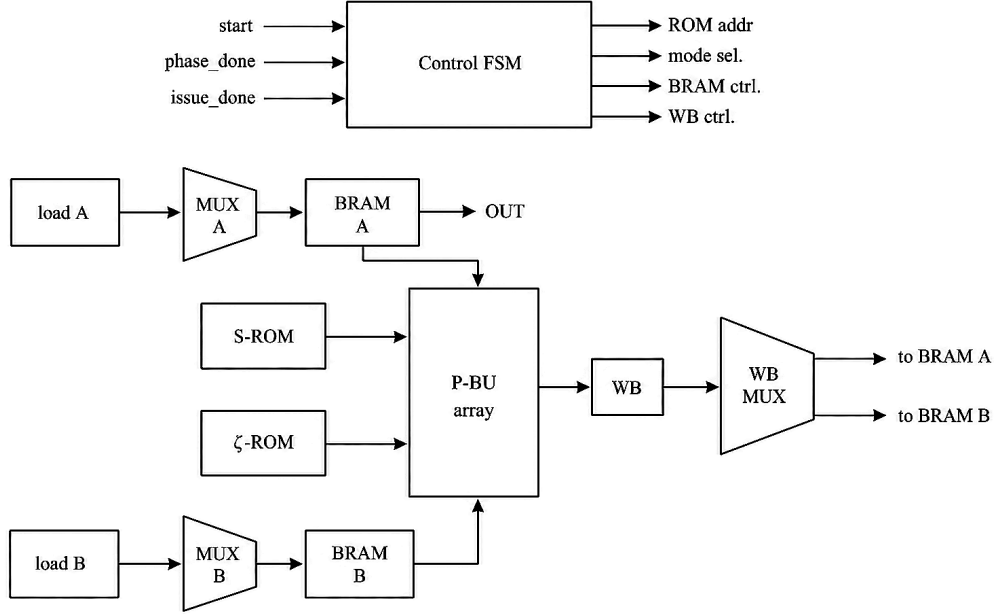


Fig. 11. Overall PMUL accelerator.

TABLE III
FPGA IMPLEMENTATION RESULTS ON QUARTUS AND VIVADO.

Flow / device	Arch.	Logic	Reg./FF	RAM/BRAM	DSP	Fmax (MHz)
Quartus / Cyclone V	4BU	3,379 ALM	3,794	24	4	125.64
Quartus / Cyclone V	8BU	10,723 ALM	8,308	48	8	111.31
Quartus / Cyclone V	16BU	17,196 ALM	16,342	96	16	107.04
Vivado / Kintex-7	4BU	4,871 LUT	2,655	12.0	8	136.41
Vivado / Kintex-7	8BU	14,837 LUT	5,200	24.0	16	118.47
Vivado / Kintex-7	16BU	26,610 LUT	10,356	48.0	32	104.10

TABLE IV
END-TO-END POLYNOMIAL MULTIPLICATION LATENCY AND THROUGHPUT.

Flow / device	Arch.	NTT(A)	NTT(B)	Pointwise	INTT+scale	Total	Time (μ s)	Throughput (k/s)
Quartus / Cyclone V	4BU	435	435	88	520	1483	11.799	84.73
Quartus / Cyclone V	8BU	307	307	56	360	1035	9.298	107.55
Quartus / Cyclone V	16BU	243	243	40	280	811	7.577	131.99
Vivado / Kintex-7	4BU	435	435	88	520	1483	10.872	91.98
Vivado / Kintex-7	8BU	307	307	56	360	1035	8.736	114.46
Vivado / Kintex-7	16BU	243	243	40	280	811	7.790	128.36

for 4BU, 8BU, and 16BU, respectively. The Vivado BRAM values include the coefficient memories and the initialized schedule/zeta ROMs loaded through `$readmemh`.

The cycle latency is unchanged between Quartus and Vivado because the two evaluations use the same cycle-level microarchitecture and the same full polynomial multiplication schedule. The time per polynomial multiplication is recomputed from the reported Fmax of each flow, so timing differences affect throughput but not the number of cycles. For example, the Vivado 16BU design completes one polynomial multiplication in $811/104.10\text{MHz} = 7.790\ \mu\text{s}$, corresponding to 128.36 thousand polynomial multiplications per second.

Table V is intended as a contextual comparison, not as an absolute ranking. The listed works differ in FPGA family, logic

primitive, memory primitive, target scheme, modulus width, and reported scope. The Kyber rows use a smaller 12-bit modulus, while the proposed designs target the 23-bit Dilithium modulus. NTT-core-only rows do not include the full sequence $\text{NTT}(A)$, $\text{NTT}(B)$, pointwise multiplication, INTT, and final scaling. For this reason, the most direct comparison in this paper is the internal 4BU/8BU/16BU scaling under the same architecture and verification flow; the prior-work rows are used to show the broader resource-latency context.

Compared with Kyber PMUL accelerators, the proposed designs require wider arithmetic and more memory resources because the Dilithium coefficient path is 23 bits wide. However, the DSP count remains controlled because reduction-constant products are implemented with shift/add/sub logic. Compared with NTT-core-only works, the proposed accelerator has a broader scope because it completes the whole polynomial multiplication operation internally and reports the end-to-end latency of the full flow.

VII. CONCLUSION

This paper presented a scalable FPGA accelerator for CRYSTALS-Dilithium polynomial multiplication. The accelerator implements $\text{NTT}(A)$, $\text{NTT}(B)$, pointwise Montgomery multiplication, INTT, and final scaling using a shared arithmetic engine. The main design contributions are a Montgomery reduction unit with one-DSP-per-lane modular multiplication, a unified CT/GS butterfly, a schedule-ROM address generator, conflict-free banked memory, and a clustered butterfly organization.

On Intel Cyclone V, the 4BU, 8BU, and 16BU designs complete one polynomial multiplication in 1483, 1035, and 811

TABLE V
IMPLEMENTATION RESULTS AND COMPARISON WITH PRIOR WORK.

Work	Platform	n	q /bits	Resource				Clock MHz	Latency (cycles)		
				LUT/ALM	REG	DSP	BRAM		NTT	INTT	PMUL
Seiler [3] ^{a,d,j}	Intel CPU	256	7681/13	–	–	–	–	–	419	394	1278
Botros [4] ^{b,d,j}	Cortex-M4	256	3329/12	–	–	–	–	–	7725	9347	27873
RISQ-V [5] ^{a,b,g,j}	Zynq-7000	256	–/16	2908	170	9	–	–	1935	1930	–
Poppelmann–Guneyusu [6] ^{a,j}	Virtex-6	256	7681/13	4549	3624	1	12	262	2096	–	–
Roy [7] ^{a,j}	Virtex-6	256	7681/13	1349	860	1	2	313	1691	–	–
Huang [8] ^{b,c,j}	Artix-7	256	3329/12	–	–	–	–	225	1834	–	–
DATE TW-4 [9] ^{b,c,f,j}	Artix-7	256	3329/12	2543	792	4	9	182	232	233	864
DATE TW-16 [9] ^{b,c,f,j}	Artix-7	256	3329/12	9508	2684	16	35	172	69	71	256
Nguyen–Tran [10] ^{b,c,e,j}	Cyclone V	256	3329/12	1401 ALM	2929	4	1	237	446	446	–
Jedlicka [11] ^{c,e,h,j}	UltraScale+	256	8380417/23	1798	2532	48	–	637	502	517	–
This work 4BU ^{c,f,h,i}	Cyclone V	256	8380417/23	3379 ALM	3794	4	24	125.64	435	520	1483
This work 8BU ^{c,f,h,i}	Cyclone V	256	8380417/23	10723 ALM	8308	8	48	111.31	307	360	1035
This work 16BU ^{c,f,h,i}	Cyclone V	256	8380417/23	17196 ALM	16342	16	96	107.04	243	280	811

^a Supports multiple n or q . ^b Kyber parameter set. ^c Optimized for constant q . ^d Software baseline. ^e NTT/INTT core only. ^f Full polynomial multiplication. ^g Processor/domain-specific accelerator. ^h Dilithium parameter set. ⁱ This work on Cyclone V. ^j Not an exact one-to-one comparison because of platform, modulus, scheme, or scope differences.

cycles, respectively. The corresponding throughputs are 84.73, 107.55, and 131.99 thousand polynomial multiplications per second. The results show that increasing the number of butterfly units improves end-to-end throughput even though the fitted Fmax decreases because of wider routing. Future work includes placement-aware routing optimization and integration into a complete Dilithium signing and verification datapath.

REFERENCES

- [1] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” FIPS 204, Aug. 2024, doi: 10.6028/NIST.FIPS.204.
- [2] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehle, “CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation,” NIST PQC Round 3 submission, 2021.
- [3] G. Seiler, “Faster AVX2 optimized NTT multiplication for Ring-LWE lattice cryptography,” Cryptology ePrint Archive, Report 2018/039, 2018.
- [4] L. Botros, M. J. Kannwischer, and P. Schwabe, “Memory-efficient high-speed implementation of Kyber on Cortex-M4,” in *Proc. AFRICACRYPT*, 2019.
- [5] T. Fritzmann, G. Sigl, and J. Sepulveda, “RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography,” *IACR Trans. Cryptographic Hardware and Embedded Systems*, pp. 239–280, 2020, doi: 10.46586/tches.v2020.i4.239-280.
- [6] T. Poppelmann and T. Guneyusu, “Towards practical lattice-based public-key encryption on reconfigurable hardware,” in *Selected Areas in Cryptography – SAC 2013*, LNCS 8282, pp. 68–85, 2014, doi: 10.1007/978-3-662-43414-7_4.
- [7] S. S. Roy, O. Reparaz, F. Vercauteren, and I. Verbauwhede, “Compact Ring-LWE cryptoprocessor,” in *Proc. CHES*, 2014, pp. 371–391, doi: 10.1007/978-3-662-44709-3_21.
- [8] Y. Huang et al., “A pure hardware implementation of CRYSTALS-KYBER PQC algorithm through resource reuse,” *IEICE Electronics Express*, 2020.
- [9] F. Yaman, A. C. Mert, E. Ozturk, and E. Savas, “A Hardware Accelerator for Polynomial Multiplication Operation of CRYSTALS-KYBER PQC Scheme,” in *Proc. DATE*, 2021, pp. 1020–1025, doi: 10.23919/DATES1398.2021.9474139.
- [10] H. Nguyen and L. Tran, “Design of Polynomial NTT and INTT Accelerator for Post-Quantum Cryptography CRYSTALS-Kyber,” *Arabian Journal for Science and Engineering*, vol. 48, no. 2, pp. 1527–1536, 2023, doi: 10.1007/s13369-022-06928-w.
- [11] P. Jedlicka, “VHDL-based Implementation of NTT on FPGA,” Student EEICT, 2021.